Étude de cas 1 : Lecture automatique des chèques

Jalon 1 : Lire le montant par positions (MLP)

chapitre 1 - réseaux de neurones

M2 SDIA - Advanced Artificial Intelligence

Année universitaire 2026–2027

Objectif du jalon

Mesurer ce qu'un perceptron multicouche sait faire de la lecture d'un montant, comprendre pourquoi il échoue et décider **quand faire confiance** à sa lecture.

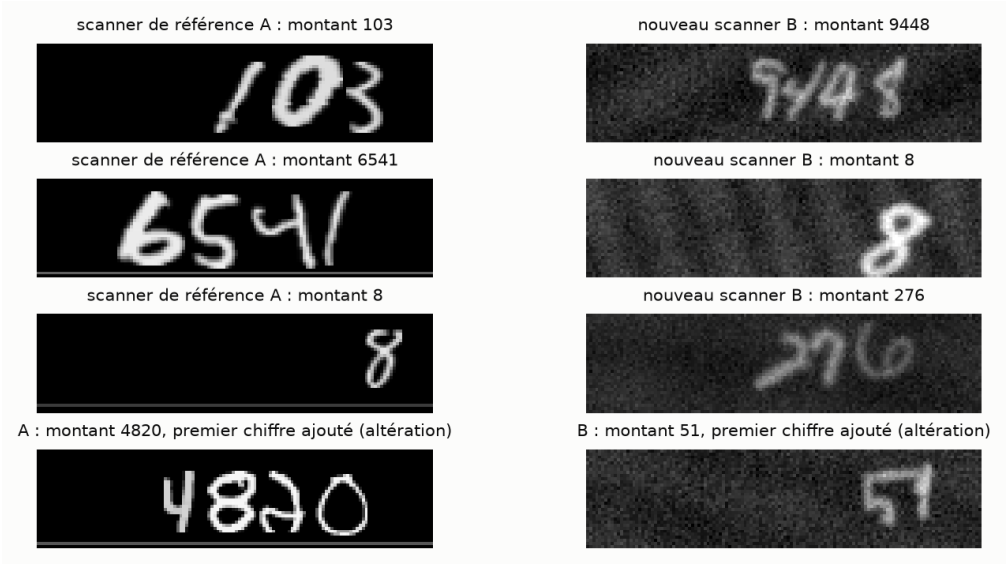
Jalon autonome : ce jalon ne suppose pas d'avoir terminé les jalons précédents. Il s'appuie uniquement sur le notebook de démarrage `uc1_demarrage.ipynb`, le dossier `data/` et le code de démarrage de la section 3. Vous pouvez repartir de votre propre travail antérieur, mais rien ne l'exige.
Environnement : Python 3, TensorFlow/Keras, scikit-learn ; Google Colab avec GPU recommandé.

1 Contexte

La banque fictive *Banque Atlas* traite chaque jour des dizaines de milliers de chèques. Le montant en chiffres, écrit à la main dans un champ cadré à droite, doit être lu automatiquement ; lorsque la machine n'est pas assez sûre d'elle, le chèque part vers un opérateur. La banque veut automatiser le plus possible **sans dépasser 1 % d'erreur** sur les chèques traités automatiquement.

Certains chèques sont **altérés** : un fraudeur ajoute un chiffre devant le montant ou retouche un chiffre existant, avec un tracé légèrement différent. Enfin, les agences reçoivent un **nouveau scanner** dont les images sont plus pâles, floues et portent un motif de sécurité ; seules 600 images de ce scanner sont étiquetées.

L'étude de cas en cinq jalons : un jalon par chapitre du cours, rendu après le chapitre correspondant, puis une synthèse défendue à l'oral en fin de module (note de décision, prédictions sur le test caché, soutenance).



2 Données et cibles de ce jalon

Fichier	Contenu
`uc1_domaineA.npz`	scanner de référence : 15 000 / 3 000 / 5 000 images (entraînement, validation, test)
`uc1_domaineB.npz`	nouveau scanner : 600 / 400 / 3 000 images

Chaque image, en niveaux de gris, mesure 32×128 pixels (encre claire sur fond sombre). Elle est accompagnée du montant écrit (chaîne de 1 à 5 chiffres), d'un indicateur d'altération et du type d'altération. Les chiffres proviennent du jeu public MNIST ; les montants, leur mise en page, les altérations et les défauts du scanner sont simulés. Les ensembles sont imposés et n'ont aucune image de chiffre en commun.

Cibles du cahier des charges concernées

Exigence	Mesure (ensemble de test)	Cible
Lire le montant (scanner A)	part des montants entièrement exacts	$\geq 0{,}85$
Automatiser (scanner A)	part des chèques traités automatiquement avec au moins 99 % de lectures exactes	$\geq 0{,}40$

Le **taux d'automatisation** se calcule en triant les chèques par confiance décroissante : c'est la plus grande part traitée automatiquement telle qu'au moins 99 % de ces lectures soient exactes (fonction `taux_automatisation` du notebook). Ces cibles décrivent le système *final* : elles ne sont pas exigées à la fin de ce jalon. Mesurez-vous par rapport à elles et expliquez l'écart ; un écart argumenté vaut mieux qu'une cible atteinte sans explication.

3 Point de départ

Le notebook de démarrage charge les données, définit l'encodage des cibles *par positions* (5 sorties softmax à 11 classes, la classe 10 signifiant « position vide »), les métriques du cahier des charges et un **MLP de référence** à cinq têtes avec son tableau de bord (section 4). C'est le modèle que ce jalon analyse, puis cherche à améliorer.

Le code ci-dessous ajoute les mesures dont vous avez besoin : exactitude par position et par longueur de montant, courbe « part automatisée contre précision », et une augmentation par décalage horizontal.

Code de démarrage (fichier `uc1_jalon1_depart.py`)

```
# Étude de cas 1 - Jalon 1 : code de démarrage
# À exécuter à la suite du notebook de démarrage, exécuté en entier.

# 1. Exactitude par position (0 = unités) et par longueur (validation)
Yv = vers_positions(A["montant_val"])
P = mlp.predict(XA["val"], verbose=0)
for k in range(L_MAX):
    print(f"position {k} :", np.mean(P[f"pos{k}"].argmax(1) == Yv[:, k]))
lus, conf = lire_positions(mlp, XA["val"])
longueur = np.char.str_len(A["montant_val"])
for n in range(1, L_MAX + 1):
    sel = longueur == n
    print(f"{n} chiffre(s) :", np.mean(lus[sel] == A["montant_val"][sel]))

# 2. Part traitée automatiquement et précision obtenue, les chèques étant
#    triés par confiance décroissante
def courbe_automatisation(correct, confiance):
    c = np.asarray(correct, float)[np.argsort(-np.asarray(confiance))]
    rang = np.arange(1, len(c) + 1)
    return rang / len(c), np.cumsum(c) / rang

part, precision = courbe_automatisation(lus == A["montant_val"], conf)
plt.plot(part, precision)
plt.axhline(0.99, color="r", ls=":")
plt.xlabel("part traitée automatiquement")
plt.ylabel("précision des lectures")
plt.show()

# 3. Augmentation : décalage horizontal aléatoire de -max_px à +max_px
#    pixels, avec remplissage par du noir
def decaler(X, max_px, rng):
    out = np.zeros_like(X)
    for i, d in enumerate(rng.integers(-max_px, max_px + 1, len(X))):
        if d >= 0:
            out[i, :, d:] = X[i, :, :X.shape[2] - d]
        else:
            out[i, :, :d] = X[i, :, -d:]
    return out

rng = np.random.default_rng(0)
X_decale = decaler(XA["train"], 4, rng)    # nouveau tirage à chaque époque
```

4 Travail demandé

Jalon 1 : Lire le montant par positions (MLP)

Notions mobilisées : perceptron multicouche, sorties multiples, softmax et entropie croisée, probabilités et confiance, régularisation, augmentation de données.

1. Analyser le MLP de référence : exactitude par position (unités, dizaines...) et par longueur de montant. Pourquoi les positions ne sont-elles pas également difficiles ?
2. Étudier la confiance (produit des probabilités maximales des cinq têtes) : courbe « part automatisée contre précision », qualité de la calibration, seuil retenu pour envoyer un chèque à l'opérateur.
3. Faire varier largeur, profondeur, dropout et taux d'apprentissage ; lire les courbes d'apprentissage et conclure sur le sur-apprentissage.
4. Tester l'augmentation par petits décalages horizontaux. Qu'apporte-t-elle au MLP, et pourquoi ?

Vérification théorique (à la main, puis vérifiée par le code)

1. Comptez les paramètres du MLP de référence (4 096 entrées, couches de 256 et 128 neurones, 5 têtes de 11 classes) et la part de la première couche. Vérifiez avec `summary()`.
2. Une tête prédit (0,6 ; 0,3 ; 0,1) sur trois classes et la bonne classe est la deuxième. Calculez la perte d'entropie croisée et le gradient par rapport aux logits.

À justifier dans votre page de décisions

Pourquoi un MLP ne peut pas être invariant par translation ; le seuil de confiance que vous retenez pour l'automatisation, et ce qu'il coûte.

5 À rendre à la fin du chapitre

Livrables du jalon 1

- le notebook du jalon, exécuté et reproductible (graine fixée, versions indiquées) ;
- le journal d'expériences à jour (fonction `journaliser` du notebook de démarrage, avec `jalon=1`), y compris les essais infructueux ;
- une page de décisions : tableau des résultats, choix retenus et justifications, réponses aux vérifications théoriques.

Protocole : tous les réglages se font sur la validation ; l'ensemble de test ne sert qu'une fois, à la fin du jalon, pour le modèle retenu. Ce jalon alimente la note de décision finale et la soutenance.